Trabajo Final — ISW-323 Taller de Base de Datos III

Caso: BusRD — Venta de boletos de autobús interurbano

1. Datos generales

Aspecto	Detalle
Modalidad	Grupos de 3 estudiantes (máximo 4). Todos los grupos hacen el mismo trabajo.
Valor	10 pts el trabajo escrito y técnico, más 5 pts la exposición (total 15 pts)
Entrega	Clase 5, antes del inicio de las exposiciones
Exposición	Clase 5, de 15 a 20 minutos por grupo, con demo en vivo
Software	SQL Server 2022 (Developer o Express) y SQL Server Management Studio (SSMS)
Unidades evaluadas	I (transacciones y concurrencia), II (indexación y optimización), III (programación, seguridad y administración)

2. Contexto del caso

BusRD es una empresa dominicana de transporte interurbano que opera rutas entre Santo Domingo y 10 ciudades del país. Vende boletos por **taquilla** y por **web**, y ambos canales escriben en la misma base de datos SQL Server.

La gerencia reporta tres problemas:

1. **Asientos vendidos dos veces.** Varios pasajeros han llegado al autobús con boleto para el mismo asiento. Ocurre cuando taquilla y web venden a la vez.
2. **Lentitud.** La pantalla de asientos disponibles, la consulta del historial de un cliente y el reporte mensual de ventas tardan demasiado.
3. **Ausencia de controles.** Todos los empleados se conectan con el mismo usuario con permisos totales, y no hay política de respaldos. Hace poco alguien borró boletos por error y no se pudieron recuperar.

Su grupo fue contratado para **diagnosticar y corregir** estos problemas. El trabajo parte de una copia de la base de datos de producción que se entrega como script.

3. Modelo de datos (base entregada)

Tabla	Descripción	Filas aprox.
Rutas	Origen, destino, distancia y precio base	20
Autobuses	Placa, tipo (Estándar 44 asientos, Ejecutivo 32 asientos)	40
Asientos	Asientos físicos de cada autobús (ventana o pasillo)	~1,640
Viajes	Salida de una ruta en una fecha con un autobús	~16,400
Clientes	Pasajeros registrados	50,000
Boletos	Asiento vendido en un viaje	~230,000
Pagos	Pago asociado a cada boleto	~225,000

Nota: La base se entrega **intencionalmente incompleta**. Solo tiene llaves primarias: no tiene llaves foráneas, restricciones CHECK, índices secundarios ni restricciones de unicidad. También contiene **datos sucios** que el grupo debe detectar.

4. Recursos entregados

Ejecutar los scripts en orden en SSMS. El script 01 tarda entre 1 y 3 minutos.

Script 01 — Creación de la base y carga de datos (01_BusRD_base.sql)

Script 02 — Procedimiento de venta actual, con el defecto (02_sp_VenderBoleto_v0.sql)

Script 03 — Las 3 consultas lentas (03_consultas_lentas.sql)

5. Entregables técnicos

E1. Integridad del modelo (1.5 pts) — Unidad I

1. Escribir consultas que **detecten** los datos sucios:
    - boletos activos duplicados (mismo viaje y asiento);
    - pagos huérfanos.
2. **Limpiar** esos datos con una regla justificada. Por ejemplo, anular el boleto duplicado vendido más tarde y documentar qué se hizo con su pago.
3. Agregar como mínimo:
    - llaves foráneas entre todas las tablas relacionadas;
    - restricciones CHECK de dominio para Estado, Canal, Metodo y Tipo, y de rango para Precio, Monto y Capacidad;
    - unicidad de Clientes.Cedula y Autobuses.Placa.
4. **Garantizar desde la base de datos** que no existan dos boletos **activos** para el mismo asiento del mismo viaje. Pista: un boleto anulado libera el asiento.

E2. Venta transaccional y concurrencia (2.5 pts) — Unidad I

1. **Reproducir el problema** con sp_VenderBoleto_v0:
    - abrir 2 ventanas de SSMS;
    - en ambas, ejecutar la venta del **mismo asiento libre** del viaje asignado, con menos de 10 segundos de diferencia;
    - evidenciar la doble venta con un SELECT.
    - Este paso debe hacerse **antes** de crear la restricción de unicidad del punto E1.4, o en una copia de la base sin ella.
2. Explicar en el informe **por qué** ocurre: qué nivel de aislamiento se usa y qué tipo de anomalía es.
3. Crear dbo.sp_VenderBoleto. Debe cumplir:

Requisito	**Detalle**
Validaciones	El viaje existe y está Programado; el asiento existe en el autobús del viaje; el cliente existe; canal y método válidos
Atomicidad	Boleto y pago se insertan en **una sola transacción**

Concurrencia	Impedir la doble venta con hints de bloqueo (UPDLOCK, HOLDLOCK) o nivel SERIALIZABLE. Deben justificar la elección
Errores	TRY…CATCH con ROLLBACK y THROW
Salida	Parámetro OUTPUT con el BoletoID generado
Demo	Parámetro @SimularDemora BIT = 0. Si vale 1, ejecuta WAITFOR DELAY '00:00:10' dentro de la transacción

4. Repetir la prueba de las 2 ventanas con el nuevo procedimiento y evidenciar que la segunda venta **espera y luego se rechaza**.
5. Crear dbo.sp_AnularBoleto (anula el boleto y registra el reverso del pago con monto negativo) dentro de una transacción.

E3. Optimización (2.5 pts) — Unidad II

Para **cada una** de las 3 consultas del script 03:

Paso	Evidencia requerida
Línea base	Plan de ejecución real (captura), lecturas lógicas y tiempo de CPU (STATISTICS IO/TIME)
Diagnóstico	Qué operador es el costoso (scan, key lookup, sort) y por qué
Corrección	Índices creados (justificar columnas clave e INCLUDE) y, si aplica, reescritura de la consulta para que sea *sargable*
Resultado	Plan y métricas después, en una tabla comparativa

Para C1, usar el viaje asignado al grupo en lugar del 16000. Para C2, usar la cédula del cliente N° grupo × 1000. Además:

- comprobar que las consultas reescritas devuelven **exactamente el mismo resultado** que las originales;
- evaluar si algún índice creado afecta el rendimiento de sp_VenderBoleto (índices vs. escrituras) y comentarlo en una conclusión breve.

E4. Seguridad (1.5 pts) — Unidad III

Se pueden crear usuarios sin login (CREATE USER … WITHOUT LOGIN) y probar con EXECUTE AS USER / REVERT.

Rol	Permisos
rol_taquilla	Solo EXECUTE sobre sp_VenderBoleto y sp_AnularBoleto, y SELECT sobre una vista vw_AsientosDisponibles. **Sin acceso directo a tablas.**
rol_reportes	SELECT sobre una vista vw_VentasMensuales. **Sin acceso** a Email ni Telefono de clientes
rol_admin	Administración completa de la base

1. Crear un usuario de prueba por rol y asignarlo.
2. Evidenciar con capturas **al menos 2 accesos permitidos y 2 denegados** por rol.
3. Explicar en el informe el principio de mínimo privilegio y por qué un procedimiento permite operar sin dar permisos sobre las tablas (*ownership chaining*).

E5. Respaldo y recuperación (1.0 pt) — Unidad III

1. Cambiar el modelo de recuperación a FULL y justificar por qué.
2. Ejecutar esta secuencia, guardando los archivos en una carpeta como C:\Backups\BusRD\:
    - backup **FULL**;
    - vender 5 boletos;
    - backup **DIFERENCIAL**;
    - vender 3 boletos;
    - backup de **LOG**;
    - anotar la hora exacta;
    - simular un error con DELETE de todos los boletos del viaje asignado;
    - backup de **LOG** de cola.
3. Restaurar en una base nueva BusRD_Recuperada hasta **un momento antes del DELETE** (STOPAT).
4. Comprobar que los 8 boletos vendidos existen y que los boletos borrados se recuperaron.
5. Proponer una política de respaldos para BusRD (frecuencia de full, diferencial y log, y retención) en una tabla.

E6. Informe y repositorio (1.0 pt)

Repositorio (GitHub o GitLab, público o con acceso al docente):

BusRD-Grupo<N>/

```
├── README.md            ← integrantes, datos asignados, orden de ejecución
├── scripts/
│   ├── 01_integridad.sql
│   ├── 02_sp_VenderBoleto.sql
│   ├── 03_sp_AnularBoleto.sql
│   ├── 04_demo_concurrencia_sesionA.sql
│   ├── 05_demo_concurrencia_sesionB.sql
│   ├── 06_indices_y_consultas_optimizadas.sql
│   ├── 07_seguridad.sql
│   └── 08_respaldo_y_restauracion.sql
└── evidencias/          ← capturas de pantalla numeradas
```

Los scripts deben **ejecutarse en orden y sin errores** sobre una base recién creada con el script 01.

Informe (PDF, máximo 10 páginas sin contar portada ni anexos):

Sección	Contenido
1. Introducción	Problema y alcance
2. Integridad	Datos sucios encontrados, limpieza y restricciones
3. Concurrencia	Reproducción, causa, solución y evidencia
4. Optimización	Tabla antes/después por consulta y conclusiones
5. Seguridad	Matriz de roles y permisos, con evidencias
6. Respaldos	Secuencia ejecutada, restauración y política propuesta
7. Conclusiones	Lecciones aprendidas y riesgos pendientes
Anexo	Aporte de cada integrante

7. Rúbrica del trabajo (10 pts)

Entregable	Excelente (100 %)	Aceptable (60 %)	Insuficiente (0–30 %)	Pts
E1 Integridad	Detecta y limpia todo, con regla justificada; todas las restricciones creadas, incluida la unicidad de boletos activos	Faltan algunas restricciones o la limpieza no está justificada	No detecta datos sucios o no hay restricciones	1.5
E2 Concurrencia	Reproduce el problema, lo explica correctamente, el procedimiento cumple todos los requisitos y la prueba lo demuestra	El procedimiento funciona pero falta la explicación o algún requisito	No reproduce el problema o el procedimiento sigue permitiendo la doble venta	2.5
E3 Optimización	Las 3 consultas tienen evidencia antes/después, mejora medible, índices justificados y el mismo resultado	2 consultas completas o justificación débil	Sin métricas o sin mejora	2.5
E4 Seguridad	3 roles correctos con evidencia de permitido y denegado, y explicación de *ownership chaining*	Roles creados pero evidencia incompleta	Acceso directo a tablas para taquilla, o sin evidencia	1.5
E5 Respaldos	Restauración STOPAT exitosa y verificada, más política propuesta	Restaura pero sin punto en el tiempo o sin política	No restaura	1.0
E6 Informe y repositorio	Completo, ordenado y los scripts corren sin errores	Scripts con errores menores o informe incompleto	No entregado o no ejecutable	1.0

8. Rúbrica de exposición (5 pts)

Criterio	Descripción	Pts
Demo en vivo	Doble venta con v0, prueba de bloqueo con el nuevo procedimiento y una consulta optimizada con su plan	2.0

Dominio técnico	Responde correctamente las preguntas del docente. Cualquier integrante puede ser interrogado	1.0
Participación	Todos los integrantes exponen una parte técnica	1.0
Claridad y tiempo	Estructura clara, de 15 a 20 minutos	1.0

La nota de exposición es **individual** dentro del rango del grupo. Un integrante que no pueda explicar su parte recibe una nota menor que el resto.

9. Reglas

- Entregas tardías: −2 pts por día. No se aceptan después de 48 horas.
- Copia entre grupos: 0 en el trabajo para todos los grupos involucrados.
- Se permite usar IA como apoyo, pero cada integrante debe poder explicar cualquier línea de los scripts durante la exposición.
- Si la demo falla por causas técnicas, se acepta un video de respaldo grabado previamente, con penalización de 0.5 pts en el criterio de demo en vivo.